

Documento delle Scelte Progettuali

Programmazione Web, Il progetto: Web Services (Caso B)

by Archonet

- **Architettura**

Il sistema migra i dati del database del Primo Progetto (AcquaFlow, MySQL su Altervista) verso un nuovo database PostgreSQL locale, tramite tre componenti indipendenti: un web-service remoto in PHP, che espone i dati di origine; una Servlet Java intermedia, che legge dal web-service remoto e scrive su quello locale; un web-service locale in Django, che riceve i dati e li salva nel nuovo database.

- **Database di destinazione: PostgreSQL**

I dati di origine sono già relazionali, con chiavi esterne ben definite tra le cinque tabelle (Cliente, PuntoFornitura, Utenza, Fattura, Lettura). Si è scelto di mantenerli relazionali anche nel database di destinazione (PostgreSQL), evitando di riprogettarli come documenti (MongoDB) senza un vantaggio pratico per l'obiettivo dell'esercizio.

- **Isolamento dal Primo Progetto**

Il web-service PHP remoto è ospitato su un sito Altervista dedicato e separato da quello del Primo Progetto, con una copia dei dati importata dal dump; anche il codice risiede in una repository distinta. Questa scelta evita qualunque rischio di interferenza con il Primo Progetto, già consegnato e in fase di valutazione. Le linee guida la consentono esplicitamente ("PHP su Altervista, o dove avete sviluppato il 1° progetto").

- **Gestione della memoria su grandi volumi di dati**

La tabella Lettura contiene circa 244.000 record. Un primo approccio, che costruiva l'intera risposta in memoria prima di inviarla, causava l'esaurimento della memoria concessa da Altervista (512 MB). La soluzione adottata applica lo stesso principio in tutta la catena: il web-service PHP scrive il JSON progressivamente, riga per riga, con query non bufferizzate; la Servlet legge la risposta allo stesso modo, tramite parsing JSON a flusso, e inoltra i dati a Django in blocchi di 500 record, invece che tutti insieme.

- **Schema del database e ruolo di Django**

Lo schema PostgreSQL è definito manualmente in uno script SQL dedicato, non generato dalle migrazioni di Django: i modelli Django puntano a queste tabelle già esistenti (opzione `managed = False`), limitandosi a leggere e scrivere senza mai alterarne la struttura. Questo evita due fonti di verità sullo schema che potrebbero disallinearsi nel tempo. Per lo stesso motivo, i valori enumerati (es. stato dell'utenza, tipologia) usano colonne testuali con vincolo CHECK invece dei tipi enumerativi nativi di PostgreSQL, risultati incompatibili con il modo in cui Django scrive i dati in blocco.

- **Adattamenti allo schema originale**

Alcuni nomi di campo sono stati uniformati allo stile del resto dello schema durante il passaggio a PostgreSQL (es. città -> `citta`, `ragSoc` -> `rag_soc`, `dataAp` -> `data_apertura`), per evitare identificatori accentati o con maiuscole interne. Il database MySQL originale resta invariato; la traduzione tra le due nomenclature è responsabilità della Servlet.

- **Autenticazione PostgreSQL: uniforme tra sistemi operativi, mai tramite sudo negli script**

I comandi amministrativi psql usano sempre una connessione con utente e password espliciti (-h localhost), identica su Windows, Mac e Linux. Nessuno script richiama mai sudo al proprio interno, per non concedere privilegi di sistema a un processo automatico.

Su Mac/Linux, dove PostgreSQL non assegna per default una password all'utente amministrativo "postgres" (a differenza di Windows, dove viene impostata durante l'installazione), è richiesto un passaggio preliminare manuale, eseguito una sola volta e con piena consapevolezza da chi installa, per impostarne una.

- **Script di installazione automatica**

Oltre al manuale passo-passo, il progetto include due script (install.ps1, install.sh) che automatizzano l'intera installazione in un solo comando; il manuale resta come riferimento e alternativa in caso di imprevisti. Due accorgimenti emersi in fase di sviluppo: da PostgreSQL 15 in poi va concesso esplicitamente il permesso di creare tabelle nello schema "public" (non più automatico); il file delle credenziali va scritto in codifica ASCII per evitare un carattere invisibile che PowerShell introduce con UTF-8. Sviluppo e collaudo: Python 3.14, Java 25, PostgreSQL 18, Tomcat 11.0.24; le versioni minime dei Prerequisiti restano quelle garantite.

- **Verifica automatica dei conteggi**

Gli script di installazione (e uno script di verifica standalone, per chi segue i passaggi manuali) confrontano al termine, tabella per tabella, il numero di record migrati con quello della fonte originale, tramite un endpoint PHP aggiuntivo (conteggio.php) che restituisce solo i conteggi, non i dati.

- **Altre scelte tecniche**

- Tomcat 11 con pacchetto jakarta.servlet (compatibile anche con Tomcat 10.1 e successivi, che condividono lo stesso pacchetto), per compatibilità con versioni recenti di Java.
- Maven Wrapper incluso nel progetto, per non richiedere un'installazione preesistente di Maven.
- bulk_create con ignore_conflicts=True lato Django, per rendere la migrazione ripetibile senza dover svuotare il database ad ogni test.
- Credenziali di accesso ai database escluse dal controllo di versione tramite .gitignore, con file di esempio pubblici che mostrano la struttura attesa
- Utente PostgreSQL dedicato (acquaflow_app) per l'applicazione, invece dell'utente amministratore predefinito: applica il principio di minimo privilegio, limitando l'accesso al solo database del progetto anche in caso di compromissione delle credenziali.